

BÁO CÁO 26-5

1. Lý do không sử dụng SCPB

Sau khi phân tích thuật toán được cài đặt trong SCPB.py, có thể thấy đây là phương pháp **Sequential Counter Pseudo-Boolean (SCPB)**, một biến thể mở rộng của Sequential Counter nhằm xử lý các **ràng buộc Pseudo-Boolean có trọng số**. SCPB đặc biệt phù hợp với các bài toán mà mỗi biến đóng góp một trọng số khác nhau vào một ràng buộc dạng:

$$\sum_{i=1}^n w_i x_i \leq K$$

Ví dụ:

$$10X_1 + 2X_2 + 6X_3 + \dots = 20$$

Tuy nhiên, việc áp dụng SCPB vào bài toán HCORAP không mang lại lợi ích và thậm chí có thể làm giảm hiệu năng hệ thống do các nguyên nhân sau:

1.1 HCORAP không có ràng buộc cứng có trọng số

Trong mô hình HCORAP, các ràng buộc cứng hiện tại chủ yếu là các ràng buộc đếm thông thường (Cardinality Constraints), trong đó tất cả trọng số đều bằng 1:

SCPB sẽ suy giảm thành **Sequential Counter chuẩn**. Trong khi đó hệ thống hiện tại đã sử dụng trực tiếp cài đặt tối ưu bằng PySAT:

```
CardEnc.atmost(..., encoding=EncType.seqcounter)
```

Việc thay thế bằng cài đặt SCPB viết bằng Python không tạo ra lợi ích về mặt lý thuyết mà còn làm tăng thời gian sinh CNF.

1.2 Không phù hợp với hàm mục tiêu

Các trọng số khác nhau trong HCORAP chỉ xuất hiện tại hàm mục tiêu. Tuy nhiên đây không phải ràng buộc cứng mà là thành phần cần tối ưu hóa:

Nếu chuyển hàm mục tiêu sang dạng SCPB, cần thêm lượng lớn biến phụ và nhiều ràng buộc cộng dồn, dẫn tới:

- Tăng mạnh số lượng biến
- Tăng số mệnh đề CNF
- Làm bộ giải SAT phải thực hiện nhiều phép suy luận không cần thiết
- Dễ gây hiện tượng timeout

Tuy nhiên, hàm mục tiêu là thứ chúng ta cần **Tối ưu hóa (Maximize/Minimize)**, chứ không phải là một phương trình ràng buộc bằng một hằng số K cố định như SCPB làm (`exactly\_k` hay `at\_most\_k`).

1.3 Totalizer hiệu quả hơn đối với ràng buộc đếm

Đối với các ràng buộc đếm:

phương pháp ITotalizer được sử dụng trong phiên bản mới có ưu thế hơn Sequential Counter nhờ:

- Cấu trúc cây nhị phân
- Khả năng lan truyền đơn vị mạnh (Unit Propagation)
- Giảm độ sâu suy luận

Do đó việc thay thế bằng SCPB không đem lại cải thiện đáng kể.

Kết luận, SCPB phù hợp với các bài toán kiểu Knapsack hoặc các hệ thống có tài nguyên trọng số khác nhau, nhưng không phù hợp với cấu trúc của HCORAP.

2. Các cải tiến và cơ chế hoạt động

Phiên bản hcorap_encoding_new.py được xây dựng từ phiên bản gốc hcorap_encoding_origin.py với nhiều cải tiến nhằm giảm kích thước không gian tìm kiếm và tăng hiệu quả giải của MaxSAT.

2.1 Thay thế Sorting Network bằng Totalizer

Phiên bản gốc sử dụng **Sorting Network (Batcher Mergesort)** để xử lý các ràng buộc đếm.

Sorting Network hoạt động bằng cách xây dựng các tầng so sánh để đưa các giá trị True lên phía trước.

Nhược điểm:

- Nhiều tầng so sánh
- Độ sâu mạch lớn
- Xung đột cần lan truyền qua nhiều lớp

Phiên bản mới thay thế bằng ITotalizer.

Totalizer xây dựng cây nhị phân cộng dồn:

Ưu điểm:

- Đảm bảo Generalized Arc Consistency (GAC)
- Lan truyền đơn vị mạnh
- Giảm chiều sâu suy luận

2.2 Hybrid AMO Constraint

Phiên bản gốc áp dụng Pairwise AMO: $O(n^2)$

với mọi kích thước.

Phiên bản mới sử dụng chiến lược lai:

- Với nhóm nhỏ: $n \leq 10$. sử dụng Pairwise AMO
- Với nhóm lớn: $n > 10$. sử dụng Sequential Counter: $O(n)$

2.3 Symmetry Breaking bằng Linear Lex-Leader

Bối cảnh triển khai

Một trong những vấn đề phổ biến khi giải các bài toán SAT/MaxSAT có tính chất phân công là hiện tượng **đối xứng nghiệm (solution symmetry)**. Đối xứng xảy ra khi tồn tại nhiều nghiệm khác nhau về mặt biểu diễn nhưng thực chất lại tương đương về mặt ý nghĩa bài toán.

Trong HCORAP, điều này xuất hiện khi nhiều nhân viên có:

- Tập kỹ năng giống nhau
- Mức độ tương đồng với các dịch vụ giống nhau
- Lịch rảnh giống nhau
- Số giờ làm tối đa giống nhau

Giả sử có hai nhân viên:

- Alice
- Bob

với:

$$\begin{aligned}Availability(Alice) &= Availability(Bob) \\ Skill(Alice) &= Skill(Bob)\end{aligned}$$

Hai nhân viên này hoàn toàn có thể hoán đổi vị trí cho nhau mà không làm thay đổi giá trị hàm mục tiêu.

Ý tưởng của Symmetry Breaking

Mục tiêu của Symmetry Breaking không phải là thay đổi tập nghiệm mà là:

chỉ giữ lại một đại diện duy nhất cho mỗi lớp nghiệm tương đương.

Phiên bản hcorap_encoding_new.py sử dụng kỹ thuật:

Linear Lex-Leader Constraint

Phiên bản cải tiến sử dụng kỹ thuật **Linear Lex-Leader Constraint**, trong đó một thứ tự nhân tạo được áp đặt lên các nhân viên tương đương.

Thay vì cho phép mọi cách hoán vị, thuật toán yêu cầu các vector phân công phải tuân theo thứ tự từ điển (Lexicographical Order).

Giả sử hai nhân viên A_1 và A_2 có vector phân công:

$$\begin{aligned}Y_{A_1} &= (y_{A_1, S_1}, y_{A_1, S_2}, \dots, y_{A_1, S_m}) \\ Y_{A_2} &= (y_{A_2, S_1}, y_{A_2, S_2}, \dots, y_{A_2, S_m})\end{aligned}$$

Thuật toán áp đặt:

$$Y_{A_1} \geq_{lex} Y_{A_2}$$

Điều này có nghĩa:

- So sánh phần tử đầu tiên của hai vector.
- Nếu hai phần tử bằng nhau thì tiếp tục so sánh phần tử tiếp theo.
- Quá trình tiếp tục cho đến khi xuất hiện phần tử khác nhau đầu tiên.
- Tại vị trí khác nhau đầu tiên, vector của A_1 phải lớn hơn hoặc bằng vector của A_2 .

Nhờ vậy chỉ giữ lại một đại diện duy nhất trong tập các nghiệm đối xứng.

Tác động đến hiệu năng

Symmetry Breaking không làm thay đổi nghiệm tối ưu của bài toán.

Nó chỉ thay đổi cách bộ giải duyệt không gian tìm kiếm.

Lợi ích chính:

1. Giảm số lượng nhánh phải duyệt
2. Giảm số lần backtracking
3. Giảm số xung đột
4. Tăng hiệu quả Unit Propagation
5. Giảm hiện tượng Symmetry Thrashing

2.4 Tích hợp Implied Constraints

Bốn Implied Constraints mạnh nhất được đưa vào hệ thống:

IC1 – Agent Capacity Bound

Giới hạn số công việc tối đa dựa trên số khung thời gian khả dụng:

Ý tưởng: Một nhân viên không thể làm số lượng dịch vụ nhiều hơn số khung giờ mà họ rảnh. Ví dụ: Nhân viên B có số giờ làm việc tối đa (Max Hours) là 8 tiếng, nhưng hôm nay họ chỉ rảnh có 3 khung giờ.

Cơ chế: Chặn trên (Upper Bound) thực tế của nhân viên B không phải là 8, mà là $\min(8, 3) = 3$. Ta thêm một Ràng buộc đếm (Sequential Counter) ép tổng biến $y_{\{B,s\}} < 3$.

Sức mạnh: Nếu không có IC1, máy tính có thể phân công cho B 4 dịch vụ. Nó sẽ tốn thêm 10 phút mò mẫm cách xếp 4 dịch vụ này vào 3 khung giờ (biến x), nhận ra là không thể (UNSAT), rồi mới quay lui. Có IC1, máy tính sẽ báo lỗi ngay từ lúc định giao dịch vụ thứ 4 ở mức độ biến y .

IC2 – Time Slot Capacity Bound

Ý tưởng: Tại một thời điểm T (ví dụ 8h00 sáng), tổng số dịch vụ được thực hiện không thể vượt quá tổng số nhân viên đang online lúc 8h00 sáng.

Cơ chế: Ta đếm số nhân viên rảnh lúc T , giả sử là 5 người. Ta thêm một Ràng buộc đếm ép tổng số biến bao phủ dịch vụ $su_{\{s,T\}} < 5$.

Sức mạnh: Tránh kích bản solver gom 10 dịch vụ nhét vào lúc 8h00 sáng rồi phát hiện ra công ty không có đủ người. Nó chặn đứng "nút thắt cổ chai" về tài nguyên ngay từ cấp độ phân bổ thời gian.

IC3 – Singleton Feasibility Pruning

Ý tưởng: Nếu dịch vụ s yêu cầu một kỹ năng đặc biệt hoặc một khung giờ khắc nghiệt mà trong toàn bộ công ty *chỉ có duy nhất 1 nhân viên A* đáp ứng đủ điều kiện (kỹ năng $r > 0$ và rảnh đúng giờ đó).

Cơ chế: Thay vì để máy tính phân vân "Có nên giao cho A hay không?", ta thêm một mệnh đề đơn (Unit Clause) ép cứng $y_{\{A,s\}} = \text{True}$.

Sức mạnh: Thuật toán Unit Propagation sẽ tự động gán biến này ngay từ giây số 0, khóa chặt các ràng buộc liên quan. Mọi nhánh tìm kiếm rẽ sang hướng $y_{\{A,s\}} = \text{False}$ đều bị triệt tiêu ngay lập tức.

IC6 – Pairwise Incompatibility Cuts

Ý tưởng: Giả sử ta định giao dịch vụ X và dịch vụ Y cho nhân viên C. Tuy nhiên, khi nhìn vào lịch, nhân viên C chỉ có thể làm X lúc 9h00 và cũng chỉ có thể làm Y lúc

9h00 (Tập hợp khung giờ khả thi của X và Y trùng khít nhau và chỉ có 1 slot duy nhất). Rõ ràng C không thể phân thân làm cả hai.

Cơ chế: Cài đặt một mệnh đề 2-SAT loại trừ lẫn nhau: $\neg y_{\{C,X\}}$ or $\neg y_{\{C,Y\}}$ (Nhân viên C không thể làm X VÀ Y cùng lúc).

Sức mạnh: Cung cấp cho Solver một mạng lưới "Pairwise Clique" (Đồ thị xung đột). Khi Solver quyết định giao X cho C, cơ chế Unit Propagation tự động gán $Y = \text{False}$ cho C. Điều này giúp dập tắt hàng vạn tổ hợp phân công sai lầm trước khi chúng kịp lan xuống mức độ biến phân bổ thời gian x.

3. Kết quả thực nghiệm

Thực nghiệm được thực hiện trên ba bộ dữ liệu và so sánh giữa phiên bản gốc (origin) với phiên bản cải tiến (new).

Instance	Phiên bản	Variables	Hard Clauses	Soft Clauses	Time (s)
instance_40_10_4_1	Origin	27191	96917	957	42.69
	New	30397	102163	957	30.62
instance_40_10_4_24	Origin	33933	128908	1093	26.76
	New	35574	125266	1093	20.02
instance_30_10_4_1	Origin	16293	60040	664	7.71
	New	15005	49718	664	7.84

Tính tỷ lệ cải thiện:

Instance	Giảm thời gian
instance_40_10_4_1	28.27%
instance_40_10_4_24	25.19%
instance_30_10_4_1	-1.68%

Kết quả cho thấy:

- Trên các bài toán kích thước lớn (40_10_4_x), phiên bản mới giảm thời gian khoảng **25–28%**.
- Trên bài toán nhỏ (30_10_4_1), sự khác biệt không đáng kể.
- Mặc dù một số trường hợp số lượng biến tăng, thời gian giải vẫn giảm do:
 - Unit Propagation hiệu quả hơn
 - Không gian tìm kiếm được cắt tỉa mạnh
 - Symmetry Breaking loại bỏ nhiều nghiệm đối xứng

Như vậy, cải tiến không tập trung vào việc giảm số biến hoặc số mệnh đề tuyệt đối mà tập trung vào **nâng cao chất lượng cấu trúc CNF**, từ đó cải thiện hiệu quả của bộ giải MaxSAT.